MiniGo Compiler

This document contains some notes about our MiniGo Compiler project.

1. Technical details and implementation choices 1
   1.1. Custom calling conventions 1
   1.2. Special display based on type in `fmt.Print` 1
   1.3. Default values for variables 1
2. Difficulties encountered 2
   2.1. Boolean comparison 2
   2.2. Variables declaration 2
   2.3. Initial variables values 3
   2.4. Identifiers and structure field access (reading and writing) 3
   2.5. Problems with the `Rewrite` module 3
3. Some extra tests 4
4. A few optimizations 4
   4.1. Grouping identical static strings 4
   4.2. Static checking of division by zero 5
   4.3. Expression simplification and constant folding 5

1. Technical details and implementation choices

1.1. Custom calling conventions

For the code generation part, we decided to use some custom calling conventions:

- the return value of a function is passed in the `rdi` register (instead of the `rax` one): a function call is like an expression, and the result of a compiled expression is always stored in `rdi`;
- we did not use callee-saver or caller-saver registers: we implemented functions at the end, and as we never use some registers, we decided not to use specific ones for functions.

1.2. Special display based on type in `fmt.Print`

To be accurate with Go printing function, we display values with a specific format, based on their types:

- `int` and `string`: display the value either with the string format `%lld` or `%s`;
- `boolean`: the boolean value is compared with the value zero to print the string `true` or `false` (and not 0 or 1 as `%b` does);
- `pointer`: the pointer is compared with the value zero (representing null pointer) to either print its address (string format `%p`) or the string `<nil>`;
- `nil`: display the string `<nil>`.

We also created the function `insert_spaces_for_print` that inserts a space between two arguments of `fmt.Print` if neither of them is a string.

1.3. Default values for variables

In Go, variables that are not explicitly initialized get a default value, based on their types. In order to match this behavior, we had two solutions:

- use the `calloc` function to zero the heap-allocated blocks (works only for heap-allocated data, i.e. structures and pointers-referenced variables);
- generate additional MiniGo code after variables declaration to explicitly initialize the variables.

We chose to use the latter, as it is more general: it works for both stack and heap allocated variables, and it allows to explicitly choose the default value (with `calloc`, we are limited to the equivalent of a zero value for all types).

To implement this, we needed to add a rewriting stage to the compiler, that adds an assign statement after each variable declaration statement. This stage must occur after typing (because we need to know the type of the variable to know its default value). In practice, we implemented this stage during typing, as a rewrite on the base AST, just

before typing a block. This way, the typing of variable assignment can be the same for the generated code than for the regular one. An example of the rewriting stage is provided below.

Original code

```
type T struct { a int; b string }
// ...
var x bool
var y T
var p1, p2, p3 = new(string), new(int), new(T)
```

After typing stage

```
var x
x = false
var y
y.a, y.b = 0, ""
var p1, p2, p3
p1, (* p1), p2, (* p2), p3, p3.a, p3.b = new(string), "", new(int), 0, new(T), 0, ""
```

We took advantage of the multiple assignments to generate a single statement per declaration. These multiple assignments will be rewritten before the code generation stage by the provided `Rewrite` module.

2. Difficulties encountered

2.1. Boolean comparison

During the compilation of boolean comparisons, we had some incoherent results when compared values were bigger than 255 (1-byte). For example, the following program output wrong comparisons:

```
fmt.Print("0 == 255: ", 0 == 255, "\n") // prints false
fmt.Print("0 == 256: ", 0 == 256, "\n") // prints *true*
```

The problem was that our comparison (instruction `set %dil`) only set the lowest least significant byte of `rdi` to zero. But to read the result of the comparison, we were using *all* the 4-bytes of the register. Therefore, if a value larger than 1 byte had been previously stored in the register `rdi` (for example 256), then the result of the boolean expression was not entirely set to zero and then mis-used for the rest of the program.

To fix this error, we added another assembly instruction to keep only the last bit of register `rdi`.

```
# compare the two values
cmpq %r8 %rdi
# set the last byte of rdi with the result (0 or 1)
set %dil
# isolate the last byte of rdi to only have 0 or 1 as result
andq $1 %rdi
# use rdi and compare it with 0
...
```

2.2. Variables declaration

When compiling variable declaration inside a block, we first naively allocated space on the stack for each variable on the block. This solution worked fine until we had variable declarations inside a loop: in that case, a 8-bytes space was allocated on the stack at each iteration (without ever being de-allocated), and resulting in a stack overflow if the loop had to many iterations. To fix this error, we discussed about two choices:

- set up a stack frame for each block, using a chosen specific register (e.g. `r11`) to play the role of the frame pointer (instead of `rbp`, already used for functions): this way, each 8-bytes space allocated for a variable inside a block is freed at the end of the block;
- only allocate memory for each variable once at the beginning of a function: a variable has therefore the same offset on the stack and this space is de-allocated at the end of the function.

We opted for the second option because it was easier to implement and less error-prone. Thus, we created the function `find_all_vars`, called before compiling a function, that recursively scans the body of a function and allocates each variable on the stack once and for all.

2.3. Initial variables values

When moving from typing to code generation, we realized that the `TEVars` statement did not contain the initial values that were given when variables were initialized with an explicit value. For instance, a statement like `var x = 2` was just transformed to `var x` by the typing stage, and the initial value (`2`) was lost in the process.

We solved this problem by adding an assignment statement after each variable declaration. For instance, a declaration like `var x = 2` is expanded in two statements: `var x; x = 2`. This step is performed in the same stage described in Section 1.3.

2.4. Identifiers and structure field access (reading and writing)

We had a bit of a struggle while implementing code generation for reading and writing memory locations (i.e. identifiers and structures fields), especially when dealing with pointers.

We finally succeeded by splitting the code generation process in two functions. Each generates code that, in the end, puts something in the `rdi` register:

- `compile_expr` generates code that puts the **value** of the expression in the `rdi` register;
- `compile_left_value` generates code that puts the **address** of the expression in the `rdi` register. This is only sound when the expression is indeed a left-value, which has been already checked during the typing stage.

This way, to read a variable, we just need to generate code with `compile_left_value` to get its address, and then we read this address from memory to get its value (and the same goes for writing).

Additionally, for structure fields access (e.g. `s.a`), we needed to make a special case for pointers: in Go, the syntax is the same for accessing a structure field on pointers and variables (both use `s.a`, no matter if `s` is a pointer or a variable).

2.5. Problems with the `Rewrite` module

While implementing the code generation, we encountered two problems with the provided `Rewrite` module (file `rewrite.ml`):

- We had a first problem when rewriting nested functions calls, for instance:

```
package main

func step(a, b int) (int, int) { return b, a+b }
func step2(a, b int) (int, int) { return b, a+b }

func main() { _, x := step(step2(step(0, 1))); }
```

  which is rewritten as

```
// ...
func main() {
  {
    var _, x
    {
      var aux1, aux2
      {
        var aux3, aux4
        step2(0, 1, (& aux3), (& aux4))
        step2(aux3, aux4, (& aux1), (& aux2))
      }
      step(aux1, aux2, (& _), (& x))
    }
  }
}
```

We see here that, in the rewritten code, the `step2` function is called two times, while it is only called once in the original code. The problem is easily fixed by changing the function given in the recursive call, line 190 in `rewrite.ml`:

```
-    let vl', e' = many rw f el' in
+    let vl', e' = many rw g el' in
```

- We had a second problem when using multiple assignments with structures, for instance:

```
type T struct { a int; b int }
// ...
var t T
t.a, t.b = 0, 0
fmt.Print(t.a, t.b)
```

which is rewritten as

```
var aux1
aux1 = new(T)
{
    var aux2
    aux2 = 0
    var aux3
    aux3 = 0
    t.a = aux2
    t.b = aux3
}
fmt.Print((* aux1).a, (* aux1).b)}}
```

Here, we can see that the old name `t.a` is still used in the assign statement (but was correctly rewritten in the call to `fmt.Print`). The problem is easily fixed by also rewriting the left part of assignment statements, line 155 in `rewrite.ml`:

```
-    let assign code lv tmp = make (TEassign ([ lv ], [ ident tmp ])) tvoid :: code in
+    let assign code lv tmp =
+      make (TEassign ([ expr rw lv ], [ ident tmp ])) tvoid :: code
+    in
```

3. Some extra tests

In order to test the behavior of our compiler, we used four extra tests alongside the provided test suite. All four tests are joined to the project and are presented below:

- `comparison_to1.go`: first test for boolean comparison, to ensure that all binary operations for comparison are correct and to immediately detect the error if comparison is in fault;
- `comparison_to256.go`: second test for boolean comparison with numbers larger than 1-byte;
- `default_struct.go`: test for structures declaration and initialization with all default values;
- `result5.go`: test for nested function with different names to correct the `Rewrite` module;
- `optimizer.go`: test for code generated with our optimization (to ensure that optimization does not break some basic arithmetics)

4. A few optimizations

4.1. Grouping identical static strings

Our first implementation naively generated one label and one string for each argument of `fmt.Print` present in the source program (as well as format strings for the underlying call to `printf`). This resulted in some identical strings being present several times in the data section (especially format strings like `"%lld"` and `"%s"`, or recurrent string as `"\n"`).

To improve this, we added a hash table of all the static strings. Each time the compiler needs a static string, it first checks this hash table to see whether it already exists, and in that case uses it instead of creating a new one.

4.2. Static checking of division by zero

During the typing phase, we added a check to detect divisions by the constant `0`, to print an error message and avoid compiling a trivially-failing program. If optimization is enabled, this is also detected for expressions that reduce to the constant `0` (see Section 4.3).

4.3. Expression simplification and constant folding

As we had a little extra time after finishing the project, we added an optimization on the boolean and integer arithmetic expressions. This optimization has two targets :

- expressions that can be trivially simplified, e.g. `x * 0` or `x || true`;
- expressions that can be reduced to one constant known at compile-time, e.g. `5 + 3` (constant folding).

To implement this, we rely on e-graphs provided by the `Ego` OCaml library. The first target was implemented using some rewriting rules to perform an equality saturation followed by a lowest-cost term extraction, and the second target was implemented with an analysis on the e-graph.

This is implemented in the `Optirw` module and can be enabled by passing the `--optimize` flag to our compiler.

An example of the supported rewrites is presented below.

Before optimization (after typing and rewrite stages)

```
func main() {
  var b
  b = false
  fmt.Print(b, "b")
  fmt.Print((! (! b)), "b")
  fmt.Print((b || true), "true")
  fmt.Print((b && true), "b")
  fmt.Print((b || false), "b")
  fmt.Print((b && false), "false")
  var a
  a = false
  fmt.Print(((! a) && (! b)), "!(a || b)")
  fmt.Print(((! a) || (! b)), "!(a && b)")
  fmt.Print((true && (false || (true && (! false)))), "true")
  fmt.Print((((! (! false)) || true) && ((false && (! (! true))) || false)), "false")
  var i
  i = 0
  fmt.Print(i, "i")
  fmt.Print((- (- i)), "i")
  fmt.Print((i + 0), "i")
  fmt.Print((i - 0), "i")
  fmt.Print((i * 1), "i")
  fmt.Print((i / 1), "i")
  fmt.Print((i * 0), "0")
  var j, k
  {
    var aux1
    aux1 = 0
    var aux2
    aux2 = 0
    j = aux1
    k = aux2
  }
  fmt.Print(((k * i) + (k * j)), "k * (i + j)")
  fmt.Print(((((1 / 2) + (3 * 4)) - 3) + ((9 % 2) * 7)), "16")
  fmt.Print(
    ((((i + 10) * (2 * k)) + ((j - 0) * 1)) - (((- (- 2)) + (3 * 2)) / ((i + 2) + 1))),
    "((j + ((i + 10) * (2 * k))) - (8 / (3 + i)))")
}
```

After optimization stage

```
func main() {
  var b
  b = false
  fmt.Print(b, "b")
  fmt.Print(b, "b")
  fmt.Print(true, "true")
  fmt.Print(b, "b")
  fmt.Print(b, "b")
  fmt.Print(false, "false")
  var a
  a = false
  fmt.Print((! (a || b)), "!(a || b)")
  fmt.Print((! (a && b)), "!(a && b)")
  fmt.Print(true, "true")
  fmt.Print(false, "false")
  var i
  i = 0
  fmt.Print(i, "i")
  fmt.Print(i, "i")
  fmt.Print(i, "i")
  fmt.Print(i, "i")
  fmt.Print(i, "i")
  fmt.Print(i, "i")
  fmt.Print(0, "0")
  var j, k
  {
    var aux1
    aux1 = 0
    var aux2
    aux2 = 0
    j = aux1
    k = aux2
  }
  fmt.Print((k * (i + j)), "k * (i + j)")
  fmt.Print(16, "16")
  fmt.Print(
    ((j + ((i + 10) * (2 * k))) - (8 / (3 + i))),
    "((j + ((i + 10) * (2 * k))) - (8 / (3 + i)))"
  )
}
```